

Thinking Right About Problem Solving – Iterative vs. Recursive Approaches

Analysis and Design of Algorithms



How Most Students Think About Problems?



The Iterative Mindset:

"I need to do something multiple times, so I'll use a loop"

- **Step-by-step execution**
Focus on explicit control flow
- **Manual state management**
Track counters and variables
- **Bottom-up construction**
Build solutions incrementally



Framing Problems the Right Way

What makes a problem algorithmic?

How do we know if recursion might help?



Tips & Tricks



Russian Doll Trick

- When you reach tiniest doll → base case
- Then you close them one by one → returning values



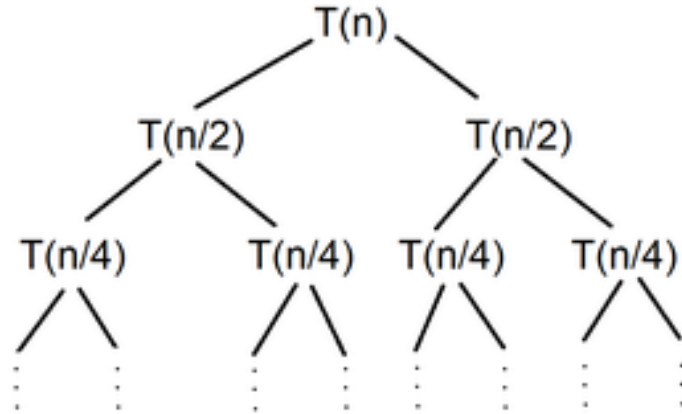
“Pretend it Works” hack

“You don't need to know how your recursive call works, just trust it will.”



“Pretend it Works” hack

Before touching the keyboard, grab a pen. **Draw** the recursion tree or timeline of calls



Top-Down Detective Trick

- I know how many ways to climb 2 stairs
- Then how many for 3? $\rightarrow \text{ways}(3) = \text{ways}(2) + \text{ways}(1)$

Detective story analogy

“Each recursion call solves a smaller mystery that unlocks the big case.”



Turn loops into stories

Every while loop is secretly a recursive drama .. It just hides its memory in variables instead of the stack



Think
Right



The Three Laws of Recursion

1. Base Case

Every recursive function must have a base case that stops the recursion

2. Progress Toward Base

Each recursive call must make progress toward the base case

3. Trust the Recursion

Assume the recursive call works correctly for smaller inputs



“Algorithmic thinking is not about memorizing patterns, it’s about learning how to shrink, solve and scale”



The Iterative Mindset:

"sum of 1 to n = n + sum(1 to n-1)"

```
n = int(input("Enter a number: "))
```

```
total = 0
```

```
i = 1
```

```
while i <= n:
```

```
    total += i
```

```
    i += 1
```

```
print("Sum from 1 to", n, "=", total)
```

```
def recursive_sum(n):
```

```
    if n == 1:
```

```
        return 1
```

```
    return n + recursive_sum(n - 1)
```

```
n = 5
```

```
print("Sum from 1 to", n, "=", recursive_sum(n))
```



The Iterative Mindset:

"sum of 1 to n = n + sum(1 to n-1)"

N = 5

5+4+3+2+1=?

Sum(5) → sum(4)

Sum(4) → Sum (3)

Sum(3) → sum(2)

Sum(2) → sum(1)

Sum(1) → sum(0) → base case

1+2=3

3+3=6

6+4=10

10+5=15



A Different Question:

"Can I break this problem into a smaller version of itself?"

- **Problem decomposition**
Break into self-similar subproblems
- **Top-down thinking**
Define solution in terms of itself
- **Trust the abstraction**
Assume subproblems are solved



Example 1: Computing Factorial

$$1*2*3*4*5 = 120$$

Fact(5)

Fact(5) $\rightarrow$ fact(4)

Fact(4) $\rightarrow$ fact(3)

Fact(3) $\rightarrow$ fact(2)

Fact(2) $\rightarrow$ fact(1)

Fact(1) $\rightarrow$ fact(0) $\rightarrow$ Base-case

$$1*2=2$$

$$2*3=6$$

$$6*4=24$$

$$24*5=120$$



Example 1: Computing Factorial

Iterative Approach (Loop)

```
def factorial_iterative(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

Recursive Approach

```
def factorial_recursive(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial_recursive(n -  
1)
```



Example 2: Sum of Array Elements

Arr=[2,4,6,8] =20

Sum([2,4,6,8]) $\rightarrow$ sum(2,[4,6,8])

Sum([4,6,8]) $\rightarrow$ sum(4,[6,8])

Sum([6,8]) $\rightarrow$ sum(6,[8])

Sum([8]) $\rightarrow$ sum(8,[])

Sum([]) $\rightarrow$ 0 Base case

0+8=8

8+6=14

14+4=18

18+2=20



Example 2: Sum of Array Elements

Iterative Thinking

```
def sum_iterative(arr):  
    total = 0  
    for num in arr:  
        total += num  
    return total
```

Recursive Thinking

```
def sum_recursive(arr):  
    if len(arr) == 0:  
        return 0  
    return arr[0] +  
        sum_recursive(arr[1:])
```



Question: Which approach is more
"natural" to explain to someone?



Example 3: Reversing a String

"Hello" , "Analysis"

"Hello" $\rightarrow$ "olleH"

"Hello" $\rightarrow$ H+ello

Ello $\rightarrow$ e+llo

Llo $\rightarrow$ l+lo

Lo $\rightarrow$ l+O

o $\rightarrow$ o+""



Example 3: Reversing a String

Iterative Solution

```
def reverse_iterative(s):  
    result = ""  
    for i in range(len(s) - 1, -1, -1):  
        result += s[i]  
    return result
```

Or with a while loop:

```
def reverse_while(s):  
    result = ""  
    i = len(s) - 1  
    while i >= 0:  
        result += s[i]  
        i -= 1  
    return result
```

Recursive Solution

```
def reverse_recursive(s):  
    if len(s) == 0:  
        return ""  
    return reverse_recursive(s[1:]) + s[0]
```



Notice: Recursion naturally expresses the "structure" of the problem



The Ultimate Question to Ask:
"Can I express the solution to this problem in
terms of solutions to smaller versions of the
same problem?"
If YES → Think recursively first!

